# فصل ۶ — جمع‌بندی و نتیجه‌گیری

## مرور مسیر طی‌شده و مقایسه‌ی نهایی

---

## ۶.۱ مقدمه

در این پژوهش، ما مسیری طولانی و پرچالش را برای حل مسئله‌ی **پیش‌بینی ریزش (Churn Prediction)** در پلتفرم Stack Overflow طی کردیم. مسئله‌ی اصلی ما فراتر از یک پیش‌بینی ساده بود؛ ما به دنبال ایجاد تعادلی ظریف میان **دقت پیش‌بینی (Accuracy)** و **قابلیت تفسیر (Interpretability)** بودیم.

در محیط‌های تجاری و عملیاتی، صرفاً دانستن اینکه "کاربر X ریزش می‌کند" کافی نیست. مدیران و تیم‌های مارکتینگ نیاز دارند بدانند **چرا** این اتفاق می‌افتد تا بتوانند مداخله کنند. ما از روش‌های ساده‌ی آماری و قوانین شفاف شروع کردیم و گام‌به‌گام به سمت مدل‌های پیچیده‌ی یادگیری عمیق حرکت کردیم تا ببینیم کجا می‌توانیم بهترین تعادل را پیدا کنیم.

این فصل مروری جامع بر کل این مسیر، تحلیل دستاوردهای هر مرحله و مقایسه‌ی نهایی روش‌ها ارائه می‌دهد.

---

## ۶.۲ مرور تحلیلی فصل‌ها

### ۶.۲.۱ فصل ۲: مهندسی داده و Event Log

سنگ بنای این پژوهش، تبدیل داده‌های خام و پراکنده به یک ساختار قابل تحلیل بود.

- **چالش:** داده‌های Stack Overflow در جداول مختلف (Posts, Comments, Votes) پخش بودند و ارزش یکسانی نداشتند. یک پاسخ تخصصی نباید هم‌وزن یک لایک ساده در نظر گرفته می‌شد.
- **راه‌حل:** تمام کنش‌های کاربر به صورت رویدادهای زمانی استاندارد `(User, Time, Action, Gain)` مدل‌سازی شد.
- **نوآوری کلیدی:** به جای وزن‌دهی دستی (Heuristic)، از **XGBoost Feature Importance** استفاده کردیم. با آموزش مدل‌های One-vs-Rest برای هر پرسونا، سیستم به صورت خودکار یاد گرفت که کدام فعالیت‌ها برای کدام گروه از کاربران "ارزشمندتر" و متمایزکننده هستند.

### ۶.۲.۲ فصل ۳: سیستم تفسیری Rule-based

در این فصل، هدف ما ساخت یک "خط‌کش" شفاف بود. سیستمی که بتوان منطق آن را در یک جمله توضیح داد.

- **قاعده:** $\text{Churn}(u) = WA(u, T_p) < A_p$
- **منطق:** اگر مجموع فعالیت وزن‌دار (WA) کاربر در بازه‌ی زمانی $T_p$ کمتر از آستانه‌ی $A_p$ باشد، سیستم هشدار ریزش می‌دهد.
- **شخصی‌سازی:** پارامترهای $T_p$ و $A_p$ ثابت نبودند؛ بلکه برای هر **پرسونا** به صورت جداگانه بهینه شدند.

- **دستاورد:** این مدل ساده توانست برای بسیاری از پرسوناها عملکرد قابل قبولی (F1 ≈ 0.30 - 0.50) داشته باشد. مزیت اصلی آن **شفافیت کامل (White-box)** بود؛ ما دقیقاً می‌دانستیم چرا یک کاربر برچسب ریزش گرفته است.

### ۶.۲.۳ فصل ۴: مقابله با Temporal Drift

با تحلیل دقیق‌تر نتایج، متوجه شدیم که "زمان" دشمن مدل‌های ثابت است. رفتار کاربران در سال ۲۰۱۲ با ۲۰۲۳ تفاوت بنیادین داشت (Temporal Drift).

- **مشکل:** پارامترهای ثابت در طول زمان دچار افت عملکرد می‌شدند.
- **راه‌حل:** معرفی **Enhanced WA** که فرمول ساده‌ی جمع را با مؤلفه‌های دینامیک جایگزین کرد:
  - **مولفه Trend:** با مقایسه‌ی نیمه‌ی اول و دوم پنجره زمانی، کاربرانی که فعالیتشان رو به افول بود جریمه می‌شدند.
  - **مولفه Recency:** با استفاده از توابع نمایی، فعالیت‌های اخیر وزن بیشتری نسبت به فعالیت‌های قدیمی گرفتند.
- **نتیجه:** این تغییرات باعث بهبود پایداری مدل شد، اما همچنان محدودیت‌های مدل‌های خطی در درک الگوهای پیچیده پابرجا بود.

### ۶.۲.۴ فصل ۵: مدل LSTM v2

برای عبور از محدودیت‌های مدل‌های خطی و شکستن سقف عملکرد، به سراغ یادگیری عمیق رفتیم.

- **مدل:** استفاده از شبکه‌های **LSTM (Long Short-Term Memory)** که توانایی "یادگیری از توالی" را دارند. برخلاف مدل‌های قبلی که فقط "مجموع" را می‌دیدند، LSTM ترتیب و فاصله‌ی زمانی رویدادها را درک می‌کرد.
- **معماری Multi-Task:** برای حفظ تفسیرپذیری، مدل را مجبور کردیم همزمان سه خروجی تولید کند:
  1. **احتمال ریزش (Churn Probability):** خروجی اصلی.
  2. **آستانه‌ی بهینه ($A_p$):** مدل حدس می‌زند این کاربر خاص چه سطح فعالیتی باید داشته باشد.
  3. **پنجره‌ی زمانی بهینه ($T_p$):** مدل حدس می‌زند چه بازه‌ی زمانی برای این کاربر مهم است.
- **دستاورد:** مدل LSTM توانست الگوهای پنهانی و غیرخطی (مثل الگوهای Burst-Pause یا تغییر رفتار در دوران COVID) را شناسایی کند و F1 Score را به طور چشمگیری افزایش دهد.

---

## ۶.۳ مقایسه‌ی نهایی روش‌ها

در جدول زیر، سه رویکرد اصلی توسعه داده شده در این پژوهش را از مناظر مختلف مقایسه می‌کنیم. این جدول نشان می‌دهد که چگونه با افزایش پیچیدگی، دقت افزایش می‌یابد اما تفسیرپذیری کاهش پیدا می‌کند.

| ویژگی | Rule-based (Simple) | Enhanced WA | LSTM (Deep Learning) |
|---|---|---|---|
| **نوع مدل** | White-box (جعبه سفید) | Grey-box (جعبه خاکستری) | Black-box (جعبه سیاه)* |
| **ورودی اصلی** | مجموع فعالیت (Scalar) | ویژگی‌های مهندسی شده (Trend/Recency) | توالی خام رویدادها (Sequence) |
| **میانگین F1** | 0.45~ | 0.55~ | **0.72 - 0.68~** |
| **قابلیت تفسیر** | ✅ عالی (White-box) | ⚠️ متوسط (Grey-box) | ❌ کم (Black-box)* |
| **حساسیت به زمان** | ❌ ندارد (ایستا) | ✅ دارد (Recency Weight) | ✅✅ عالی (Sequence Learning) |
| **مدیریت Drift** | ضعیف (نیاز به تنظیم دستی) | متوسط (پارامترهای پویا) | عالی (یادگیری Context) |
| **هزینه‌ی محاسباتی** | بسیار پایین (ثانیه) | پایین (دقیقه) | بالا (ساعت‌ها با GPU) |

> * **نکته:** اگرچه LSTM ذاتاً Black-box است، اما معماری Multi-Task ما با خروجی دادن $A_p$ و $T_p$ تلاش کرد تا حدی این نقیصه را جبران کند و پلی به دنیای تفسیرپذیر بزند.

---

## ۶.۴ نتیجه‌گیری کلیدی و پیشنهاد استراتژیک

۱. **پارادوکس دقت و تفسیرپذیری:**

- هیچ مدل "کاملی" وجود ندارد. اگر اولویت سازمان **توضیح‌دهی به ذی‌نفعان** است، سیستم **Rule-based** یا **Enhanced WA** بهترین انتخاب است. اما اگر اولویت **شناسایی حداکثری** کاربران در معرض خطر است، **LSTM** بی‌رقیب است.

۲. **اهمیت Context و پویایی:**

- رفتار کاربران ثابت نیست. مدل‌هایی که **Temporal Drift** (تغییرات زمانی) و **Persona** (تفاوت نقش‌ها) را نادیده می‌گیرند، در محیط‌های واقعی به سرعت کارایی خود را از دست می‌دهند. شخصی‌سازی پارامترها برای هر پرسونا یک ضرورت است، نه یک انتخاب.

۳. **پیشنهاد نهایی: رویکرد ترکیبی (Hybrid Approach):**

- بهترین معماری برای محیط عملیاتی، استفاده‌ی همزمان از هر دو دنیاست:
  1. **لایه تشخیص (Detection):** استفاده از **LSTM** برای پایش تمام کاربران و شناسایی دقیق و زودهنگام کاربران پرخطر.
  2. **لایه توضیح (Explanation):** استفاده از **Enhanced WA** برای تولید "توجیه" یا "Reasoning" برای کاربرانی که توسط LSTM شناسایی شده‌اند (مثلاً: "این کاربر پرخطر است چون فعالیت اخیرش (Recency) افت شدید داشته است").